Atividade de ordenação manual – Estrutura de Dados
Carlos Eduardo Souza Oliveira

-BUBBLE SORT:

V E R D E O L I V A B R A S I L

V E R D E O L I V A B R A S I L ---------- Compara os elementos adjacentes e troca a ordem se o primeiro for maior.
E V R D E O L I V A B R A S I L
E R V D E O L I V A B R A S I L
E R D V E O L I V A B R A S I L
E R D E V O L I V A B R A S I L
E R D E O V L I V A B R A S I L
E R D E O L V I V A B R A S I L
E R D E O L I V V A B R A S I L
E R D E O L I V V A B R A S I L
E R D E O L I V A V B R A S I L
E R D E O L I V A B V R A S I L
E R D E O L I V A B R V A S I L
E R D E O L I V A B R A V S I L
E R D E O L I V A B R A S V I L
E R D E O L I V A B R A S I V L
E R D E O L I V A B R A S I L V ---------- Fim da primeira iteração, cada uma garante a ordenação dos últimos elementos.

E R D E O L I V A B R A S I L V
E R D E O L I V A B R A S I L V
E D R E O L I V A B R A S I L V
E D E R O L I V A B R A S I L V
E D E O R L I V A B R A S I L V
E D E O L R I V A B R A S I L V
E D E O L I R V A B R A S I L V
E D E O L I R V A B R A S I L V
E D E O L I R A V B R A S I L V
E D E O L I R A B V R A S I L V
E D E O L I R A B R V A S I L V
E D E O L I R A B R A V S I L V
E D E O L I R A B R A S V I L V
E D E O L I R A B R A S I V L V
E D E O L I R A B R A S I L V V ---------- Fim da segunda iteração. Dois últimos elementos ordenados

-SELECTION SORT:

V E R D E O L I V A B R A S I L

V E R D E O L I V A B R A S I L ---------- Compara todos os elementos, pega o menor e troca com o início da parte não ordenada.
A E R D E O L I V V B R A S I L
A A R D E O L I V V B R E S I L
A A B D E O L I V V R R E S I L ---------- D já na posição certa.
A A B D E O L I V V R R E S I L
A A B D E E L I V V R R O S I L
A A B D E E I L V V R R O S I L
……..

- INSERTION SORT:
V E R D E O L I V A B R A S I L

V	- E R D E O L I V A B R A S I L
E V	- R D E O L I V A B R A S I L
E R V	- D E O L I V A B R A S I L
D E R V	- E O L I V A B R A S I L
D E E R V	- O L I V A B R A S I L
D E E O R V	- L I V A B R A S I L
D E E L O R V	- I V A B R A S I L
D E E I L O R V	- V A B R A S I L
D E E I L O R V V	- A B R A S I L
A D E E I L O R V V	- B R A S I L
A B D E E I L O R V V	- R A S I L
A B D E E I L O R R V V	- A S I L
A A B D E E I L O R R V V	- S I L
A A B D E E I L O R R S V V	- I L
A A B D E E I I L O R R S V V	- L
A A B D E E I I L L O R R S V V	- **Ordenado**

Verde: letra inserida; Amarelo: letras movidas

- SHELL SORT:
V E R D E O L I V A B R A S I L

h = 4

V E R D E O L I V A B R A S I L --------- Compara elementos com h de distância.
E E R D V O L I V A B R A S I L
E E R D V O L I V A B R A S I L
E E L D V O R I V A B R A S I L
E E L D V O R I V A B R A S I L
E E L D V O R I V A B R A S I L --------- Quando realiza uma troca, compara olhando para
E E L D V A R I V O B R A S I L trás (quando possível).
E A L D V E R I V O B R A S I L
E A L D V E B I V O R R A S I L
E A B D V E L I V O R R A S I L
E A B D V E L I V O R R A S I L
E A B D V E L I A O R R V S I L
E A B D A E L I V O R R V S I L
A A B D E E L I V O R R V S I L
A A B D E E L I V O R R V S I L
A A B D E E L I V O I R V S R L
A A B D E E I I V O L R V S R L
A A B D E E I I V O L R V S R L
A A B D E E I I V O L L V S R R --------- Fim da primeira iteração. Repete-se o mesmo com
um gap menor (1).

- MERGE SORT:
M E R G E Q U I C K H E A P

M	E	R	G	E	Q	U	I	C	K	H	E	A	P
0	1	2	3	4	5	6	7	8	9	10	11	12	13

merge_sort(inicio, meio, fim) - **SAÍDA**
ENTRADA

- merge_sort(0, 6, 13) – **A C E E E G H I K M P Q R U**
  M E R G E Q U I C K H E A P
  - merge_sort(0, 3, 6) – **E E G M Q R U**
    M E R G E Q U
    - merge_sort(0, 1, 3) – **E G M R**
      M E R G
      - merge_sort(0, 0, 1) – **E M** ---------- **Exemplo – A)**
        M E
      - merge_sort(2, 2, 3) – **G R**
        R G
    - merge_sort(4, 5, 6) – **E Q U**
      E Q U
      - merge_sort(4, 4, 5) – **E Q**
        E Q
      - merge_sort(6, 6, 6) - **U**
        U
  - merge_sort(7, 10, 13) – **A C E H I K P**
    I C K H E A P
    - merge_sort(7, 8, 10) – **C H I K** ---------- **Exemplo – B)**
      I C K H
      - merge_sort(7, 7, 8) – **C I**
        I C
      - merge_sort(9, 9, 10) – **H K**
        K H
    - merge_sort(11, 12, 13) – **A E P**
      E A P
      - merge_sort(11, 11, 12) – **A E**
        E A
      - merge_sort(13,13, 13) - **P**
        P

EXEMPLOS:
A) M E. Troca os dois se estiverem na ordem errada **(E M).**

B) C I, H K
I, H K C
I, K C H
K C H I
C H I K

-Trata um ponteiro para cada vetor e compara as letras de cada um. Adiciona a menor no vetor resultado e move-se o ponteiro correspondente.

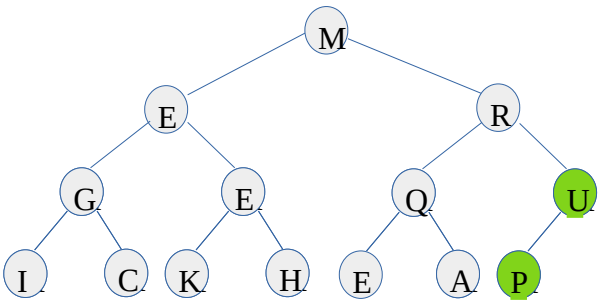
- **QUICK SORT:**
M E R G E Q U I C K H E A P

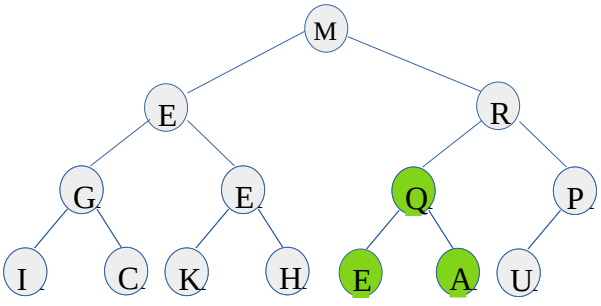
M E R G E Q U I C K H E A P ------ Se o elemento já está na posição certa, move-se seu
M E R G E Q U I C K H E A P ponteiro, senão, troca as letras ou mantém o ponteiro
M E A G E Q U I C K H E R P parado.
M E A G E Q U I C K H E R P
M E A G E Q U I C K H E R P
M E A G E E U I C K H Q R P
M E A G E E H I C K U Q R P
M E A G E E H I C K U Q R P
K E A G E E H I C M U Q R P ------ Fim da primeira iteração, todos à direita de M são
maiores que ele, e todos à esquerda são menores.

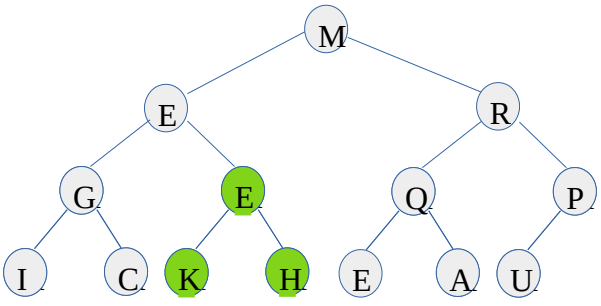
K E A G E E H I C M U Q R P
K E A G E E H I C M U Q R P
K E A G E E H I C M U Q R P
K E A G E E H I C M U Q R P
K E A G E E H I C M U Q R P
K E A G E E H I C M U Q R P
K E A G E E H I C M U Q R P
C E A G E E H I K M U Q R P ------ Fim da segunda iteração.

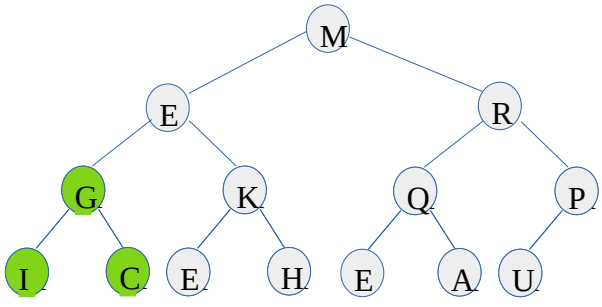
- **HEAP SORT:**
M E R G E Q U I C K H E A P

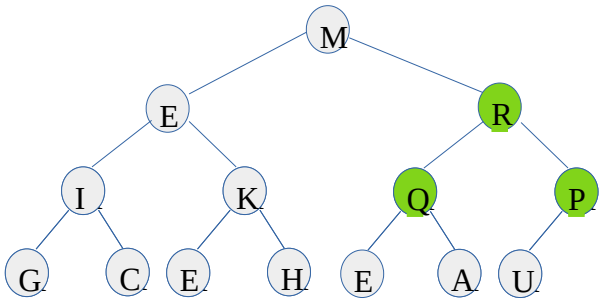
- Cria-se uma árvore com o vetor de cima para baixo, da esquerda para direita.
- Analisa-se cada subárvore da última para a primeira.

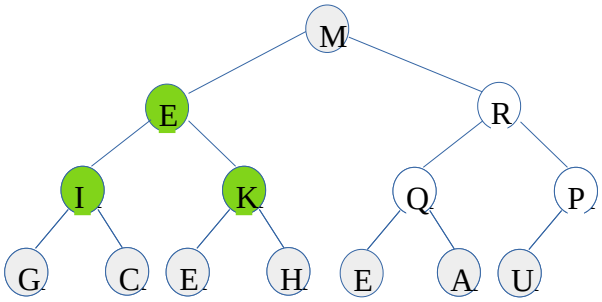


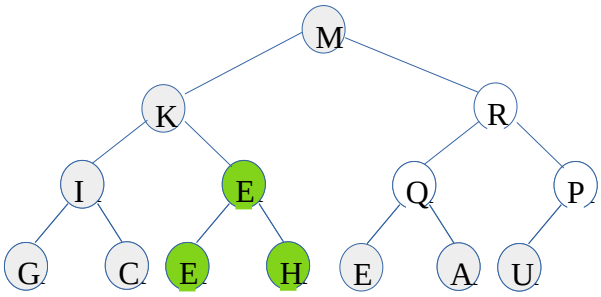


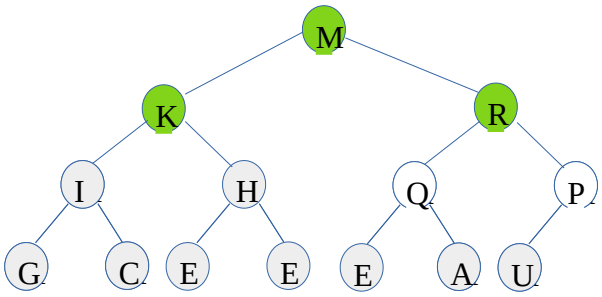


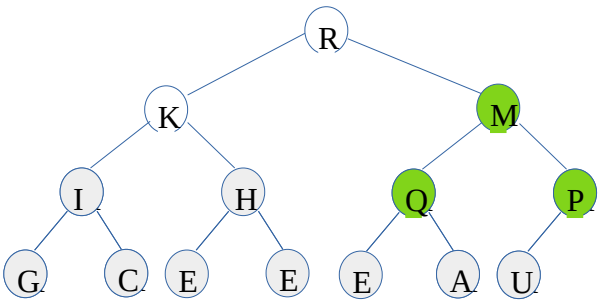


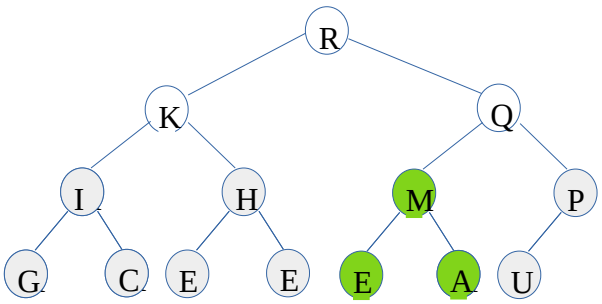


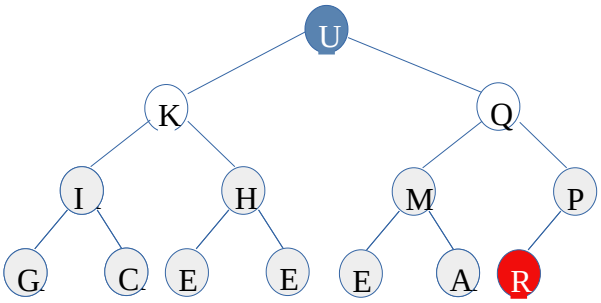


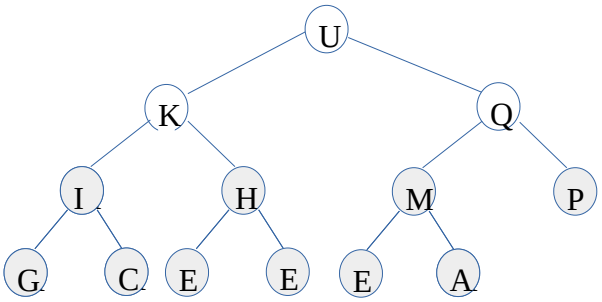












U K Q I H M P G C E E E A R -------- Fim da primeira iteração, última letra ordenada.
Repete-se o mesmo para a nova árvore, até acabar seus elementos e o vetor estiver ordenado.